Decentralized Near-Synchronous Local-First Programming Collaboration

Leon Freudenthaler
leon.freudenthaler@fh-campuswien.ac.at
FH Campus Wien/TU Wien
Vienna, Austria

Abstract

This dissertation investigates near-synchronous collaboration in programming environments by means of Conflict-free Replicated Data Types (CRDTs). While screen sharing and video conferencing offer basic features for collaborative programming, IDE-embedded collaboration tools provide precise tracking of source code changes and developer control. CRDTs enable decentralized collaboration, eliminating central server dependencies. However, existing CRDTs fail to preserve user intent and handle complex formats, leading to inefficiencies. Using a Design Science Research approach, this dissertation addresses four key research areas: (1) Identifying the limitations of existing CRDT algorithms in preserving user intent for programming files, (2) Optimizing CRDT implementations for programming collaboration requirements, (3) Determining the appropriate granularity for replicated data types in programming environments, (4) Exploring supervised imitation learning to enhance system capabilities in merging conflicts. This work aims to improve collaborative programming tools, ensuring higher consistency, efficiency, and user satisfaction.

CCS Concepts

• **Human-centered computing** → **Asynchronous editors**.

Keywords

Programming collaboration, CRDT, decentralized collaboration

ACM Reference Format:
Leon Freudenthaler. 2024. Decentralized Near-Synchronous Local-First Programming Collaboration. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '24), September 16–20, 2024, Vienna, Austria.* ACM, New York, NY, USA, 3 pages. https://doi.org/10.1145/3650212.3685555

1 Problem Statement and Relevance

Collaboration tools influenced the way teams work together, especially in remote and distributed working environments. They enable users to collaborate simultaneously on the same data. In software development asynchronous collaboration has become commonplace with version control systems like Git. However, the advent of real-time collaboration tools has introduced new dynamics, enabling remote pair programming and code reviews. Traditional tools utilized for remote pair programming encompass features such as shared screens, video conferencing, and chat functionalities. However, IDE-embedded collaboration tools take this a step further by integrating directly into the development environment, offering a more seamless and natural developer experience. They offer developers greater control, and enable precise tracking of source code changes in, from a user's perspective, real-time – technically they are implemented *near-synchronously*. They enable developers to actively contribute to code as if they where physically co-located. Several server-based platforms that enable simultaneous code collaboration emerged, for example "Code with me" (JetBrains) and "Visual Studio Live Share" (Microsoft).

At the core of such collaboration tools lies the principle of consistency. Near-synchronous collaboration tools often prioritize eventual consistency (convergence) over strong consistency due to the implications of the PACELC theorem [1]. The PACELC theorem states that if a network partition exists, a system must make a tradeoff between availability and consistency[1]. Those tools typically operate in distributed environments where network partitions and failures are prevalent, yet system availability and partition tolerance are a necessity. Consequently, loosening their consistency constraints enables the system to remain available even during network partitions, as updates can be made locally and then propagated asynchronously to other nodes.

Several algorithms emerged to realize near-synchronous collaborative editing, with Operational Transformation (OT) and Conflict-free Replicated Data Types (CRDTs) being the most prominent ones. OT, proposed in 1989 [4], has undergone various iterations, resulting in numerous research efforts [18, 23, 24]. [20] introduced two properties OT algorithms must satisfy – TP1 and TP2. However, edge cases where revealed in which proposed algorithms (dOPT [4], adOPTed [20], GOT [25], SOCT2 [22], IMOR [6], SDT [8][9]) failed to satisfy TP2 [6, 15]. Today only two reliable forms of OT still exist: server-based OT (Jupiter [12] and SOCT3/4 [27]), and TTF[14]. Server-based OTs, exemplified by their success in practical applications such as Google Docs, are accompanied by the inherent limitation of relying on a centralized server to facilitate node updates. [5] indicate that the only decentralized OT algorithm for which no invalidation is known is the TTF[14]. Randolph et al. [19] even shows that it is impossible to achieve convergence (TP2) in classic formulation of OT. Besides that, there are discussions whether TTF is *"a CRDT in disguise"* [26] and [18] argues its complexity has hindered widespread adoption, resulting in only a handful of functional implementations.

Thus, conflict-free replicated data types (CRDTs) emerged as an alternative to address the challenges, and at the same time offering

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ISSTA '24, September 16–20, 2024, Vienna, Austria
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0612-7/24/09
https://doi.org/10.1145/3650212.3685555

decentralized convergence[2]. CRDTs are capable of resolving conflicts independently on each node. This not only enhances system resilience but also facilitates the adoption of local-first applications and offline capabilities, aligning more seamlessly with the – mainly asynchronous – software development workflow. It reduces the overall latency in cases where collaborating participants are geographically closer to each other than to the central server and it enables the system to leverage various network topologies beyond client-server architectures [4]. Similarly to Git, CRDTs offer asynchronous collaboration. However, while Git requires manual resolution of conflicts, CRDTs programmatically resolve them.

At a first glance, CRDTs may seem to offer a promising approach, tackling numerous challenges inherent in OT. However, research reveals that CRDTs introduce their own set of limitations. Issues arise concerning the preservation of user intention, where contextual factors play a pivotal role. Most of existing CRDT algorithms focus on plain text and fall short in addressing the nuances of more complex formats like programming source code files.[13, 17, 21]

[10] shows how existing CRDT algorithms do not accurately preserve user intent in rich text editing. This is primarily due to CRDTs handling updates on a per-character base, which refers to the granularity at which operations are tracked and applied within the data structure. The per-character approach of most CRDTs brings an overhead, especially in handling multi-character operations common in programming workflows, such as reordering/moving ranges of items and mass deletions. Another issue that comes with CRDTs is the size of metadata that needs to be handled. Depending on the CRDT structure and algorithm used, this may increase the overall size dramatically[10].

When dealing with source code files at a textual level, additional challenges arise. Merged outcomes often exhibit inaccuracies, unpredictability, and may result in syntax errors, primarily due to the inherent semantic properties of programs[11]. One aim in that direction is presented by authors of CoAST (a conflict free replicated AST)[11]. Instead of using a string-based CRDT approach they incorporate an abstract syntax tree (AST) as their base data structure. Program source code is typically reasoned about in terms of its AST, which abstracts from syntax tokens. Designing CRDTs based on the AST enables them to better understand the underlying program structure without requiring semantic reasoning. Replicating the AST helps maintain syntactical correctness and shifts merge conflicts from character-based operations to tree-structure changes. The paper concludes with multiple limitations that need further research. The presented merge strategy lacks heuristics to determine winning operations for conflicting operations, requiring improvements like role-based ordering or user prompts for agreement. Additionally, the merging of AST happens on a per-node base, necessitating more nuanced strategies (merge granularity issue).

2 Research Questions, Methodology and Contribution

This section presents the research questions this dissertation seeks to address based on the provided context of the problem statement and its relevance. Additionally, it outlines the methodology and contributions of this dissertation in addressing the proposed research questions, along with the anticipated artefacts. The dissertation will use a Design Science Research approach as a methodology which is described in [3] and [16]. This approach involves identifying a problem, creating a new artefact to address it, evaluating the artefact's effectiveness, and iterative refining it based on feedback, with the ultimate goal of contributing both practical solutions and theoretical knowledge to the field.

(RQ1) *What are the limitations of existing CRDT algorithms in preserving user intention when handling complex data formats in the form of source code files?*
  (1) Extensive systematic literature review (based on [7]).
  (2) Analysis of presented CRDT algorithms within the context of programming environments.
  (3) Artefacts: Taxonomy technical report. Edge cases where user intent is not preserved (case studies, problem case). Benchmarks of existing CRDT algorithms using source code files.

(RQ2) *How can existing CRDT implementations be optimized to fulfill user intention and performance requirements when used in collaborative programming environments?*
  (1) Examine CRDT optimization strategies in different layers of architecture (e.g. programming language, type of CRDT, data structure used).
  (2) Artefacts: Present and discuss different optimization strategies. Measure strategies with benchmarks from RQ1.

(RQ3) *What is the appropriate level of granularity in replicated data types with regards to near-synchronous programming environments?*
  (1) Examine the suitability and applicability of replicated data types and their corresponding granularity that may better align with the requirements of programming-specific CRDTs compared to those designed for (rich) text editing, with a focus on data structures such as syntax tokens and abstract syntax trees.
  (2) Artefacts: Novel CRDT implementation designed to operate at other levels than presented CRDTs, aimed at facilitating collaborative editing of programming code with different granularity and enhanced consistency.

(RQ4) *How can supervised imitation learning or similar approaches be leveraged to investigate the feasibility of enabling systems to "learn merging" in collaborative environments, and what are the potential implications and challenges associated with this approach?*
  (1) Collecting a diverse dataset of merge conflicts and their resolutions from real-world collaborative editing scenarios.
  (2) Train a supervised imitation learning model using the annotated dataset to learn merging behaviors and decision-making processes observed in human collaborators.
  (3) Evaluation of the trained models on a validation dataset to assess performance in accurately predicting merge resolutions and handling various types of conflicts.
  (4) Artefacts: Design/specification of learning model tailored to the merging process in collaborative environments. Trained learning model. Evaluation metrics.

References

[1] Daniel Abadi. 2012. Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story. *Computer* 45, 2 (2012), 37–42. https://doi.org/10.1109/MC.2012.33

[2] Paulo Sérgio Almeida. 2023. Approaches to Conflict-free Replicated Data Types. (Oct. 2023). arXiv:2310.18220 https://arxiv.org/abs/2310.18220

[3] M. Benner-Wickner, R. Kneuper, and I. Schlömer. 2020. *Leitfaden für die Nutzung von Design Science Research in Abschlussarbeiten*. IUBH Internationale Hochschule. https://books.google.at/books?id=dABRzgEACAAJ

[4] C. A. Ellis and S. J. Gibbs. 1989. Concurrency control in groupware systems. *SIGMOD Rec.* 18, 2 (jun 1989), 399–407. https://doi.org/10.1145/66926.66963

[5] Victor B. F. Gomes, Martin Kleppmann, Dominic P. Mulligan, and Alastair R. Beresford. 2017. Verifying strong eventual consistency in distributed systems. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 109 (oct 2017), 28 pages. https://doi.org/10.1145/3133933

[6] Abdessamad Imine, Pascal Molli, Gérald Oster, and Michael Rusinowitch. 2003. Proving Correctness of Transformation Functions in Real-Time Groupware. 277–293. https://doi.org/10.1007/978-94-010-0068-0_15

[7] Barbara Kitchenham and Stuart Charters. 2007. Guidelines for performing Systematic Literature Reviews in Software Engineering. 2 (01 2007).

[8] Du Li and Rui Li. 2004. Preserving operation effects relation in group editors. In *Proceedings of the 2004 ACM Conference on Computer Supported Cooperative Work* (Chicago, Illinois, USA) *(CSCW '04)*. Association for Computing Machinery, New York, NY, USA, 457–466. https://doi.org/10.1145/1031607.1031683

[9] Du Li and Rui Li. 2008. An Approach to Ensuring Consistency in Peer-to-Peer Real-Time Group Editors. *Comput. Supported Coop. Work* 17, 5–6 (dec 2008), 553–611. https://doi.org/10.1007/s10606-005-9009-5

[10] Geoffrey Litt, Sarah Lim, Martin Kleppmann, and Peter van Hardenberg. 2022. Peritext: A CRDT for Collaborative Rich Text Editing. *Proc. ACM Hum.-Comput. Interact.* 6, CSCW2, Article 531 (nov 2022), 36 pages. https://doi.org/10.1145/3555644

[11] Aäron Munsters, Angel Luis Scull Pupo, and Jens Nicolay. 2022. COAST: A Conflict-free Replicated Abstract Syntax Tree. In *17th International Conference on Software Technologies (Proceedings of the 17th International Conference on Software Technologies - ICSOFT, Vol. 1)*. Scitepress, 187–196. https://doi.org/10.5220/0011278800003266

[12] David A. Nichols, Pavel Curtis, Michael Dixon, and John Lamping. 1995. High-latency, low-bandwidth windowing in the Jupiter collaboration system. In *Proceedings of the 8th Annual ACM Symposium on User Interface and Software Technology* (Pittsburgh, Pennsylvania, USA) *(UIST '95)*. Association for Computing Machinery, New York, NY, USA, 111–120. https://doi.org/10.1145/215585.215706

[13] Brice Nédelec, Pascal Molli, Achour Mostéfaoui, and Emmanuel Desmontils. 2013. LSEQ: an Adaptive Structure for Sequences in Distributed Collaborative Editing. *DocEng 2013 - Proceedings of the 2013 ACM Symposium on Document Engineering*, 37–46. https://doi.org/10.1145/2494266.2494278

[14] Gérald Oster, Pascal Molli, Pascal Urso, and Abdessamad Imine. 2006. Tombstone Transformation Functions for Ensuring Consistency in Collaborative Editing Systems. *International Conference on Collaborative Computing: Networking, Applications and Worksharing* 0 (11 2006). https://doi.org/10.1109/COLCOM.2006.361867

[15] Gérald Oster, Pascal Urso, Pascal Molli, and Abdessamad Imine. 2006. Proving correctness of transformation functions in collaborative editing systems. (04 2006).

[16] Ken Peffers, Tuure Tuunanen, Marcus Rothenberger, and S. Chatterjee. 2007. A design science research methodology for information systems research. *Journal of Management Information Systems* 24 (01 2007), 45–77.

[17] Nuno Preguica, Joan Manuel Marques, Marc Shapiro, and Mihai Letia. 2009. A Commutative Replicated Data Type for Cooperative Editing. In *2009 29th IEEE International Conference on Distributed Computing Systems*. 395–403. https://doi.org/10.1109/ICDCS.2009.20

[18] Aurel Randolph, Hanifa Boucheneb, Abdessamad Imine, and Alejandro Quintero. 2013. On Consistency of Operational Transformation Approach. *Electronic Proceedings in Theoretical Computer Science* 107 (Feb. 2013), 45–59. https://doi.org/10.4204/eptcs.107.5

[19] Aurel Randolph, Hanifa Boucheneb, Abdessamad Imine, and Alejandro Quintero. 2015. On Synthesizing a Consistent Operational Transformation Approach. *IEEE Trans. Comput.* 64, 4 (2015), 1074–1089. https://doi.org/10.1109/TC.2014.2308203

[20] Matthias Ressel, Doris Nitsche-Ruhland, and Rul Gunzenhäuser. 1996. An integrating, transformation-oriented approach to concurrency control and undo in group editors. In *Proceedings of the 1996 ACM Conference on Computer Supported Cooperative Work* (Boston, Massachusetts, USA) *(CSCW '96)*. Association for Computing Machinery, New York, NY, USA, 288–297. https://doi.org/10.1145/240080.240305

[21] Hyun-Gul Roh, Myeongjae Jeon, Jinsoo Kim, and Joonwon Lee. 2011. Replicated abstract data types: Building blocks for collaborative applications. *J. Parallel Distrib. Comput.* 71 (03 2011), 354–368. https://doi.org/10.1016/j.jpdc.2010.12.006

[22] Maher Suleiman, Michèle Cart, and Jean Ferrié. 1997. Serialization of concurrent operations in a distributed collaborative environment. In *Proceedings of the 1997 ACM International Conference on Supporting Group Work* (Phoenix, Arizona, USA) *(GROUP '97)*. Association for Computing Machinery, New York, NY, USA, 435–445. https://doi.org/10.1145/266838.267369

[23] Chengzheng Sun. 2010. Operational transformation: Research and application. In *The 2010 14th International Conference on Computer Supported Cooperative Work in Design*. I–I. https://doi.org/10.1109/CSCWD.2010.5472013

[24] Chengzheng Sun and Clarence Ellis. 1998. Operational transformation in real-time group editors: issues, algorithms, and achievements. In *Proceedings of the 1998 ACM Conference on Computer Supported Cooperative Work* (Seattle, Washington, USA) *(CSCW '98)*. Association for Computing Machinery, New York, NY, USA, 59–68. https://doi.org/10.1145/289444.289469

[25] Chengzheng Sun, Xiaohua Jia, Yanchun Zhang, Yun Yang, and David Chen. 1998. Achieving convergence, causality preservation, and intention preservation in real-time cooperative editing systems. *ACM Trans. Comput.-Hum. Interact.* 5, 1 (mar 1998), 63–108. https://doi.org/10.1145/274444.274447

[26] David Sun, Chengzheng Sun, Agustina Ng, and Weiwei Cai. 2020. Real Differences between OT and CRDT in Correctness and Complexity for Consistency Maintenance in Co-Editors. *Proc. ACM Hum.-Comput. Interact.* 4, CSCW1, Article 21 (may 2020), 30 pages. https://doi.org/10.1145/3392825

[27] Nicolas Vidot, Michèle Cart, Jean Ferrié, and Maher Suleiman. 2000. Copies convergence in a distributed real-time collaborative environment. 171–180. https://doi.org/10.1145/358916.358988

Received 2024-07-07; accepted 2024-07-22